

Assignment – Module 2: Understanding Large Language Models (LLMs)

Name	SAGAR KUMAR
PRN	SOE25BTAM29
Section	A
Institute	Intellipaat School of Technology, Pune
Subject	Generative AI
Submission	Soft copy (LMS)

Q1. Understanding Why an LLM Is Called “Large”

a) **Model B** is a Large Language Model. It is trained on a huge amount of text and has billions of parameters.

b) What “large” means

“Large” refers to two things:

1. **Large training data:** billions or trillions of words from books, websites, code and articles.
2. **Large number of parameters:** billions of adjustable numbers inside the model.
(It also means a large amount of computing power is needed to train it.)

c) **How parameters help learning**

Parameters are like **small knobs or settings** inside the neural network. During training, the model guesses the next word. If the guess is wrong, the parameters are adjusted a little. After this repeats billions of times, the parameters slowly **store the patterns of language:** grammar, word meanings, facts and writing styles.

d) **Why more parameters capture more complex patterns**

More parameters mean **more memory and more ways to connect ideas**. A small model may learn simple things, such as “the sky is ___ → blue”. A large model can also learn deeper patterns, such as tone, logic, code structure, translation and long-distance links between sentences. It is like a small notebook versus a big library. The bigger one can hold more detailed knowledge.

Q2. From Text to Tokens

Input: “Artificial intelligence is changing software development.”

a) What is a token?

A token is a **small piece of text** that the model reads. It can be a full word, part of a word, or a punctuation mark.

Example: ["Artificial", " intelligence", " is", " changing", " software", " development", "."] (a long word may split further, such as “develop” + “ment”).

b) Why convert to tokens?

Computers do not understand letters and words like humans. They only work with **numbers**.

Tokens give a fixed, manageable **vocabulary**, roughly 50,000–200,000 pieces, so the model can handle any text, including new or rare words, by breaking them into known pieces.

c) Why give each token a numerical ID?

Each token gets a number (e.g., “software” – 3241). The model uses this ID to look up an **embedding**, a list of numbers that represents the token’s meaning. Math can then be done on these numbers inside the neural network.

d) Same tokens in two sentences

If two sentences share tokens (e.g., “software” in both), the model uses the **same ID and the same meaning vector** each time. So what it learned about “software” in one sentence helps it understand the other. This helps the model see **common patterns** and relationships between sentences.

Q3. Understanding Next-Token Prediction

Prompt: “The student submitted the assignment before the ____”

a) “deadline” (72%) is the most likely token.

b) Core working principle

An LLM works by **predicting the next token** using probabilities. It looks at all the previous words and calculates how likely each possible next word is. It then chooses one. It does not “know” the answer like a human; it picks what is **most likely based on patterns** it learned.

c) What happens next?

The chosen token is **added to the input**. Now the text is “The student submitted the assignment before the deadline”. The model then predicts the next token again (e.g., “.” or “and”).

d) A paragraph is repeated next-token prediction

The model writes **one token at a time**. Each new token is added to the text, and the model predicts again. This loop continues until a full sentence, paragraph or essay is complete, or a stop token is reached. A long answer is simply this step repeated hundreds of times.

Q4. Pre-Training vs. Fine-Tuning/RLHF

a) The **first stage** (learning from a huge amount of text) is **pre-training**.

b) The **second stage** (making it helpful, safe and direct) is **fine-tuning / RLHF** (Reinforcement Learning from Human Feedback).

c) **What pre-training gives**

General knowledge of language and the world: grammar, vocabulary, facts, reasoning patterns and coding styles. It is like a student who has read the whole library.

d) **What fine-tuning/RLHF solves**

A pre-trained model only continues text. It may ramble, ignore questions, or give unsafe answers. Fine-tuning teaches it to **follow instructions**. In RLHF, humans rank the model's answers, and the model learns to give answers people find **helpful, honest and safe**.

e) **Why both are needed**

- Without **pre-training**, the model has no knowledge and cannot write well.
 - Without **fine-tuning/RLHF**, the model has knowledge but does not behave like a good assistant.
- Together they produce a model that is **smart and well-behaved**, which is what a chatbot like ChatGPT needs.

Q5. Identifying LLM Capabilities

Task	a) Capability	b) Why an LLM is suitable	c) Difficulty for rule-based system
1. 20-page document – 5 points	Summarisation	Understands meaning and picks the most important ideas	Rules cannot judge which sentences are “important”; meaning is spread across pages
2. Informal – professional email	Rewriting / tone change	Learned many writing styles and can change tone while keeping meaning	Slang, short forms and casual language have endless variations
3. Explain a hard concept simply	Explanation / simplification	Can rephrase ideas, give examples and adjust to the audience	Needs real understanding of the concept; rules cannot create new examples
4. Messy transcript – action items	Information extraction / structuring	Can find who does what and by when, even in unclear	People speak in broken sentences, interrupt each other,

Task	a) Capability	b) Why an LLM is suitable	c) Difficulty for rule-based system
		speech	and say dates in many ways (“next Fri”, “by EOD”)

Q6. Traditional Software or LLM?

a) **System A (fee calculator)** suits traditional software.

b) **System B (student assistant)** suits an LLM.

c) **Comparison**

Characteristic	Traditional Software (Fee Calculator)	LLM (Student Assistant)
Output	Deterministic: same input – same output every time	Probabilistic: answers can vary
Accuracy need	Must be 100% exact (money)	Approximate, helpful guidance is fine
Input type	Structured (course, category, scholarship)	Unstructured natural language questions
Logic	Fixed rules written by programmers	Patterns learned from data
Testing / explaining	Easy to test and audit	Harder to test; may hallucinate

d) **Problems if an LLM calculates fees**

- It may give a **wrong amount** (LLMs make math mistakes).
 - It may give **different answers** to the same question.
 - Errors involve **real money**, which can lead to complaints or legal issues.
 - It is **slower and more costly** than a simple formula.
 - It is hard to audit **why** a number was produced.
- The fee calculator should use normal code. The LLM can only explain the result in simple words.

Q7. One LLM, Multiple Tasks

Do I agree that five different models are needed? No.

- **General-purpose LLMs:** Modern LLMs like GPT or Gemini are **general-purpose**. One model can summarise, translate, explain code, rewrite and plan. The **prompt** tells it which task to do.
- **Patterns learned during training:** The model was trained on research papers, translations, code, emails and plans. So it already learned the patterns needed for all five tasks.
- **Multiple capabilities:** Summarising, translating and explaining are all language tasks. They all come from the same skill: understanding and generating text.
- **Narrow AI vs LLMs:** Old **narrow AI** systems were built for one job only (e.g., a spam filter or a translation-only model). Modern LLMs are **multi-task**: one model, many jobs.

Example where a specialised system is better:

Fraud detection on bank transactions, or medical image diagnosis. A specialised ML/DL model trained on that exact data is more accurate, faster, cheaper and easier to test than a general LLM. **Exact calculations**, such as the fee calculator, are another example.

Q8. Structuring Unstructured Information

a) Table

Issue	Important Details	Urgency	Suggested Action
Project submission shows “ Not Submitted ” even after upload	Tried yesterday; portal error; uploaded on 3rd attempt; status still “Not Submitted”; deadline tomorrow ; student worried about losing marks	High (deadline tomorrow)	IT/portal team to check server logs and confirm the upload; inform the faculty; give the student a temporary email-submission option or deadline protection; reply to the student today

b) Why this is structuring unstructured data

The complaint is **free-form text**. It is emotional, in story form, with no fixed format. We convert it into **fixed fields** (Issue, Details, Urgency, Action) like a database row. Now it can be sorted, searched, counted and assigned to a team easily.

c) Prompt for consistent results

You are a helpdesk assistant for a college. Read the student complaint below and convert it into a table with exactly four columns: **Issue, Important Details, Urgency, Suggested Action**.

Rules:

- Issue: one short sentence.
- Important Details: only facts from the complaint (dates, attempts, error messages,

deadline).

– Urgency: only one of **High** / **Medium** / **Low**. Use High if a deadline is within 48 hours.

– Suggested Action: 1–3 practical steps for the college staff.

– Do not add information that is not in the complaint. Write “Not mentioned” if something is missing.

Output only the table in Markdown format.

Complaint: “”””{complaint_text}””””

d) Why an LLM beats a keyword system

- Keywords miss **meaning**. “Uploaded” and “Not Submitted” appear together, and only an understanding of context shows that this is a sync problem, not a failed upload.
- Students write in **many different ways** (“portal crashed”, “site not working”, “it’s showing error”).
- An LLM can understand **urgency from context** (“deadline is tomorrow”) even without the word “urgent”.
- An LLM can **write a suggested action**; keywords can only match words.

Q9. Diagnosing LLM Limitations

Problem	a) Limitation	b) Why it happens	c) Workaround
A: Gives info on events after training	Knowledge cutoff (and hallucination)	The model only knows data up to its training date. It guesses about newer events.	Connect to web search / RAG with current data; show the cutoff date; ask for sources
B: Misses info in the middle of a long document	Context window limit / “lost in the middle”	The model can read only a limited number of tokens and pays less attention to the middle of long inputs	Chunk the document, summarise each part, then combine; use RAG to pick relevant sections
C: Wrong math with confident explanation	Weak calculation / reasoning errors	It predicts likely tokens instead of really calculating. Numbers are split into tokens oddly.	Use code execution / calculator tools ; ask for step-by-step working; verify answers
D: Links professions to one gender	Bias	Training data from the internet contains human stereotypes, and the model learns them	Careful prompts, bias testing, balanced fine-tuning data, human review, content filters

d) Most dangerous in high-stakes use: Problem C (confident wrong answers).

In banking, medicine or engineering, a wrong number such as a medicine dose, a loan amount or a load calculation can directly cause harm or financial loss. Because the explanation sounds **confident**, users may trust it and not check. Bias (D) is also very serious in hiring or loans, because it can cause unfair treatment of people at a large scale.

Q10. Hallucination Detection Challenge

a) Phenomenon: Hallucination, specifically **fabricated citations**. The AI invented papers, journals and DOIs that do not exist.

b) Why it looks convincing

The answer follows the **exact format** of real citations: proper titles, real-sounding journal names, dates and DOI-style numbers. The model learned *how citations look*, so it produced text that **looks right**. A student who trusts AI and has little time may not check it.

c) Why confident wording is not proof

An LLM always writes fluently and confidently. That is how it was trained to write. **Confidence comes from language patterns, not from checking facts**. The model has no built-in “truth-check”. A confident tone and a correct fact are two completely different things.

d) Four-step verification process

1. **Search each paper title** on Google Scholar, IEEE Xplore, arXiv or Semantic Scholar.
2. **Check the DOI** at doi.org. A fake DOI will not open.
3. **Check the author’s real profile**: university page, Google Scholar profile or ORCID, and their real publication list.
4. **Open and read the actual paper** (at least the abstract). Cite it only if it exists and supports your point. Otherwise remove it.

e) Link to Mata v. Avianca (2023, USA)

In this case, lawyers used ChatGPT for legal research and submitted a court brief citing **six court cases that did not exist**. ChatGPT had invented them, with fake names, quotes and citations. When the lawyers asked ChatGPT if the cases were real, it said “yes”. The judge **fined the lawyers \$5,000** and criticised them publicly. This is the same problem as the student’s: fake but realistic references, used without checking. The lesson is the same for students and professionals: **always verify AI-provided sources in the original database before using them**.

Q11. Improving a Poor LLM Prompt

Poor prompt: “Make a study plan for me.”

a & b) Improved prompt

Goal: I have a **Python programming interview in 10 days**. Create a day-by-day study plan to help me clear it.

Background: I am a B.Tech first-year student. I am comfortable with basic Python (variables, loops, if-else, lists).

Weak areas: **Functions, Object-Oriented Programming (OOP), and problem-solving** (coding questions).

Time available: **2 hours per day**.

Expected output: A table with columns: Day | Topic | What to study (1 hr) | Practice problems (1 hr) | Resource link. Add 3 practice questions for each day.

Constraints: Spend more time on my weak areas. Keep Day 9 for a full mock test and Day 10 for revision only. Use only free resources. Keep explanations simple.

c) Why it works better

- The AI now knows the **exact goal** (Python interview), the **time limit** (10 days × 2 hours) and **my level**.
 - It can **focus on weak areas** instead of giving a general plan.
 - A **clear output format** (table) makes the plan easy to follow.
 - Constraints prevent unnecessary or unrealistic content.
- More context leads to less guessing, which gives a more useful and personalised answer.**

d) Using the LLM iteratively

After Day 3, I tell the AI: *"I finished Days 1–3. Functions are now clear, but I'm still struggling with classes and inheritance. I could only study 1.5 hours on Day 2. Please update the plan for the remaining 7 days."*

I can also share my quiz scores and ask: *"Give me 5 more OOP questions at medium level."* This way, the plan **adjusts to my real progress**, like a personal tutor.

Q12. Working With an LLM Through an API

a) System message change

```
{"role": "system",  
  "content": "You are a helpful Python Coding Assistant. Answer only  
Python "  
          "programming questions. Explain in simple steps, give  
clean "  
          "code examples with comments, and point out common  
mistakes."}
```

b) Role of user input

The user's question is sent as a message with "role": "user" inside the messages list of the **payload**. This is the actual question the model must answer.

c) Why include the API key?

The API key **proves who is making the request**. The server uses it to check permission, track usage and bill the correct account. Without it, the server rejects the request (status **401 Unauthorized**). It must be kept secret, for example in an environment variable.

d) What to extract from the JSON response

```
reply = data["choices"][0]["message"]["content"]
```

choices is a list → [0] takes the first answer → message → content holds the text.

e) Why not read content when status ≠ 200

If the status is not 200 (e.g., 401, 429, 500), the request **failed**. The response will contain an **error message**, not choices. Trying to read data["choices"] will cause a **KeyError** and crash the program. So we must check the status first and show a friendly error.

f) One improvement

Add **error handling with try/except, a timeout and retry**:

```
try:
    response = requests.post(API_URL, headers=headers, json=payload,
                             timeout=30)
    if response.status_code == 200:
        print(response.json()["choices"][0]["message"]["content"])
    else:
        print("Error:", response.status_code, response.text)
except requests.exceptions.RequestException as e:
    print("Network problem:", e)
```

(Other ideas: keep chat history for memory, load the API key from .env.)

Q13. Job-Oriented Case Study — Designing a Reliable LLM Assistant

a) Relevant LLM limitations

Hallucination (confident wrong answers), knowledge cutoff, context window limit (very large documents), weak math, no knowledge of private company data, and privacy/data-leak risk.

b) Information newer than the model's knowledge

Use **RAG (Retrieval-Augmented Generation)**. Store company documents in a **vector database**. When an employee asks something, the system first **searches for the latest relevant documents** and gives them to the LLM, which answers from them. When documents change, update the database. There is no need to retrain the model.

c) Documents bigger than the context window

- **Chunking**: split the document into small parts (e.g., 500–1,000 tokens).
- **Retrieve only the relevant chunks** for each question (RAG).

- For summaries, use **map-reduce**: summarise each chunk, then summarise the summaries.
- Use a model with a larger context window if needed.

d) Reducing hallucinations

- Instruct: “Answer **only** from the provided documents. If not found, say *‘I could not find this in company documents.’*”
- **Show sources** (document name and section) with every answer.
- Use a **low temperature** (e.g., 0–0.3) for factual tasks.
- Run automatic checks and collect user feedback (thumbs up/down).

e) Responses needing human verification

Legal, HR, finance or salary matters; security and compliance; customer-facing emails before sending; code before deployment; policy decisions; and any answer where the source is missing or confidence is low.

f) Where code execution helps

Calculations (totals, percentages, dates), analysing data in Excel/CSV files, **running and testing code** that the assistant explains or writes, and creating charts. Real code gives **exact** results instead of guessed ones.

g) Why “Do not hallucinate” is not enough

The model **does not know when it is hallucinating**. It cannot see the difference between a true fact and a likely-sounding guess, because both come from the same next-token prediction. An instruction may reduce the problem slightly, but it cannot fix it. We need **system-level solutions**: RAG, citations, tools, verification and human review.

Q14. Job-Oriented Debugging — Python LLM API Application

a) Assumptions in the current code

1. The request will **always succeed** (no network error or timeout).
2. The status is **always 200**.
3. The response is **always valid JSON**.
4. JSON always has a **choices** key.
5. **choices** is **never empty** (index [0] exists).
6. **message** and **content** **always exist** and are not empty.

b) Why check the status code first

The status code tells whether the request **worked**. 200 = success. 400 = bad request, 401 = wrong API key, 429 = too many requests, 500 = server error. On an error, the JSON body has an error field, not choices. Checking first lets us **show the correct error message** instead of crashing.

c) Safer response handling

```

import requests

def get_reply(payload):
    try:
        response = requests.post(API_URL, headers=headers,
                                  json=payload, timeout=30)
    except requests.exceptions.Timeout:
        return "Error: The request timed out. Please try again."
    except requests.exceptions.RequestException as e:
        return f"Error: Network problem - {e}"

    if response.status_code != 200:
        return f"Error {response.status_code}: {response.text[:200]}"

    try:
        data = response.json()
    except ValueError:
        return "Error: Server did not return valid JSON."

    choices = data.get("choices")
    if not choices:
        return "Error: No 'choices' in response."

    content = choices[0].get("message", {}).get("content")
    if not content:
        return "Error: Reply content is missing."
    return content

```

d) What to print/log while debugging

Status code; response body (first part); time of the request; the request URL and model name; prompt length; the exact exception type and message; and the response time.

Never log the full API key. Mask it, e.g., sk-****abcd.

e) Stop one failure from ending the chatbot

Put the call inside the **main loop with try/except**, so an error only prints a message and the loop **continues**:

```

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    reply = get_reply(build_payload(user_input))
    print("Bot:", reply)  # error text is shown, loop continues

```

f) Two extra improvements before sharing with other developers

1. **Retry with exponential backoff** for 429/500 errors (wait 1s, 2s, 4s...), with a maximum number of retries.

2. **Store the API key in a .env file / environment variable**, not in the code.
(Also: use the logging module instead of print, write unit tests with mocked responses, and add a README.)
-

Q15. Research + Job-Oriented Question — Can Next-Token Prediction Produce Reasoning?

Interview question: “If an LLM is fundamentally predicting the next token, why does it appear capable of reasoning?”

a) What next-token prediction means

Given some text, the LLM calculates a **probability for every possible next token** and picks one. It adds that token to the text and repeats. Every answer — code, essay or maths solution — is built **one token at a time** this way.

b) How learned patterns support complex behaviour

To predict the next token well on trillions of words of text, the model must learn more than spelling. It has to learn **grammar, facts, cause-and-effect, the structure of arguments, how code runs, and how solutions are written step by step**. These patterns are stored in billions of parameters. When a new question appears, the model **combines** these patterns, so its output looks like reasoning.

c) Concrete example

Question: “A pen costs ₹10 and a notebook costs ₹25. What is the cost of 2 pens and 1 notebook?”

The model predicts tokens one after another:

“2 pens” → “= 2×10 ” → “= ₹20” → “Notebook = ₹25” → “Total = 20 + 25” → “= ₹45”.

Each step is just a likely next token, based on thousands of similar solved problems it has seen. Because each written step becomes part of the input for the next prediction, the chain of small predictions produces a **correct, intelligent-looking answer**. This is why “think step by step” (chain-of-thought) prompts help.

d) Example of a confident failure

Ask: “How many times does the letter ‘r’ appear in ‘strawberry’?”. Many models have confidently answered “2” (the correct answer is 3). This happens because the model sees **tokens** like “straw” + “berry”, not individual letters. Another example: if you change the names or numbers in a familiar maths puzzle, or add one irrelevant sentence, the model often makes mistakes while still sounding sure.

e) Why prediction does not guarantee correctness

The model is trained to produce **likely** text, not **true** text. If a wrong answer looks statistically similar to correct answers, the model may produce it. It has **no built-in fact-checker**, no direct access to the real world, and it cannot run a calculation unless it is given a tool. Also, one wrong token early in a chain can lead the whole answer in the wrong direction.

f) Conclusion: Is it “reasoning”?

It is fair to say LLMs show **useful reasoning-like behaviour**, but it is **not reliable reasoning**

in the human sense. They are excellent pattern-based problem solvers that work well on familiar problems and break on unusual ones. In practice, an AI engineer should treat LLM reasoning as **powerful but unverified**, and add tools, tests and human checks.

Research — Clearly Separated

What the module taught	An LLM's basic operation is next-token prediction using probabilities. It is trained with pre-training and fine-tuning/RLHF. It has limitations such as hallucination, knowledge cutoff, context limits and weak math.
What I found in external research	(1) Wei et al. (2022), " <i>Chain-of-Thought Prompting Elicits Reasoning in Large Language Models</i> " (NeurIPS 2022, Google Research): asking models to write intermediate steps greatly improves performance on maths and logic tasks. This shows that "reasoning" appears when the model generates step-by-step tokens. (2) Mirzadeh et al. (2024), " <i>GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models</i> " (Apple, arXiv:2410.05229): when only the names/numbers in maths problems were changed, or an irrelevant sentence was added, model accuracy dropped noticeably. This suggests the models often rely on pattern matching rather than true logical reasoning. (3) Bender et al. (2021), " <i>On the Dangers of Stochastic Parrots</i> " (ACM FAccT): language models produce fluent text without true understanding of meaning.
My own conclusion	Next-token prediction at a very large scale can imitate reasoning very well , and step-by-step generation makes it stronger. However, research shows it is fragile and can fail confidently. So I would call it " reasoning-like behaviour " and always verify important outputs.